TACON 2026 LEIPZIG
16. — 17. September
Testing, Testautomatisierung und QM in der Softwareentwicklung

Kubernetes Workloads *unter Last*

Tools, Herausforderungen und Best Practices

STEPHAN SCHWARZ

Senior DevOps Consultant

iits-consulting GmbH

Einzelne Illustrationen dieser Präsentation wurden mit KI erstellt.

about me

Stephan Schwarz
44 Jahre alt - verheiratet, zwei Kinder

Rollen in der Vergangenheit:

- Network Administrator
- Systems Engineer
- Cloud Engineer
- Infrastructure Developer
- Platform Engineer

seit 2017: **Kubernetes**

seit 2021: **Consultant**



what about you?



Es war einmal vor langer Zeit in einer weit, weit entfernten Galaxis…



Was ist Performance?

Performance = **Erwartungen werden erfüllt** an:

- Durchsatz (in bytes/s, req/s oder tx/s)
- Latenz (z.B. p95 in ms)
- Skalierbarkeit (concurrent users und Faktor)
- Ressourcenverbrauch (% cores, GB, IOPS)
- Verfügbarkeit (%, number of 9s)

Erwartungen sind individuell unterschiedlich



Warum Lasttests?

- **Stabiler Betrieb**
- Testen der **Skalierbarkeit**
- Sicherstellung von **SLAs**
- Ermittlung der benötigten **Ressourcen**
- Optimierung von **Kosten**
- Aufdecken von **Bottlenecks**



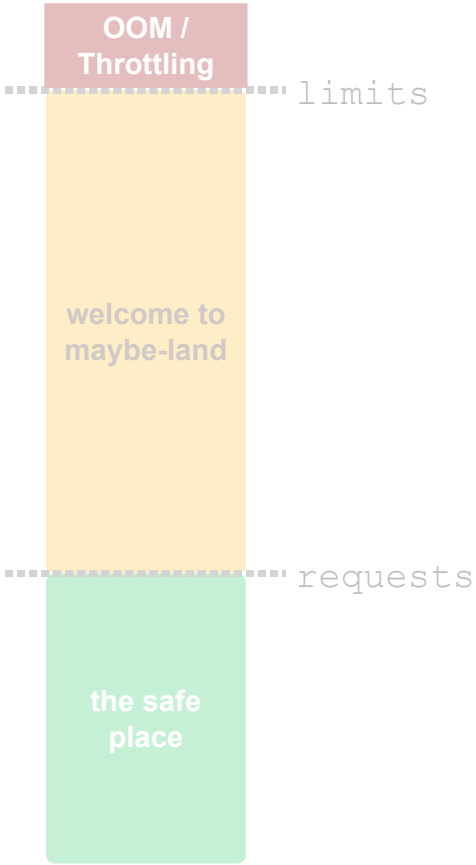
Typische Bottlenecks

CPU / RAM
Kubernetes resource requests und limits

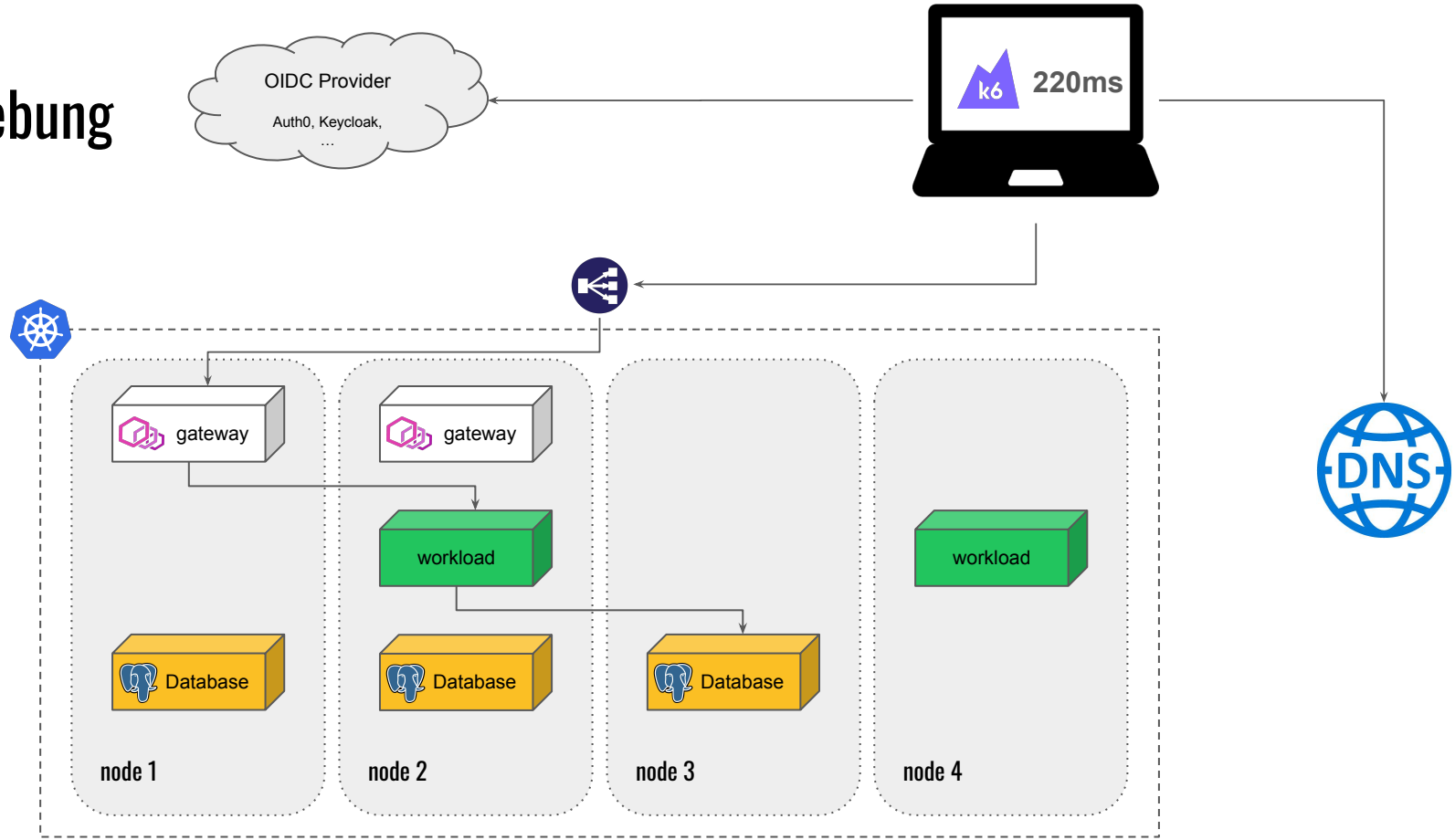
Disk I/O & Filehandles
IOPS, max. number of file handles and threads

Netzwerk
DNS, Ingress Controller, Service Mesh

Externe Abhängigkeiten (DB, S3, APIs, ...)

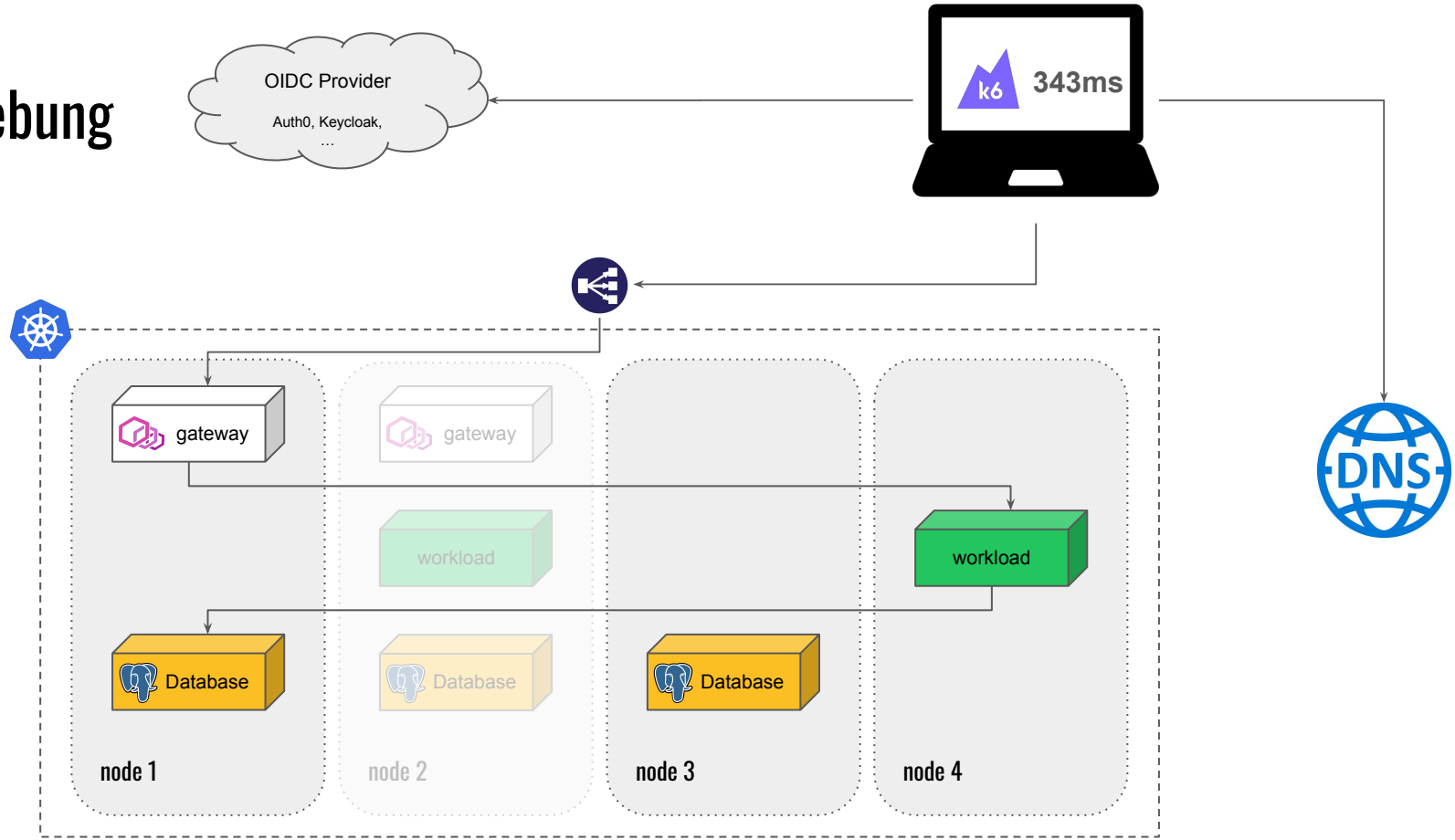


Testumgebung

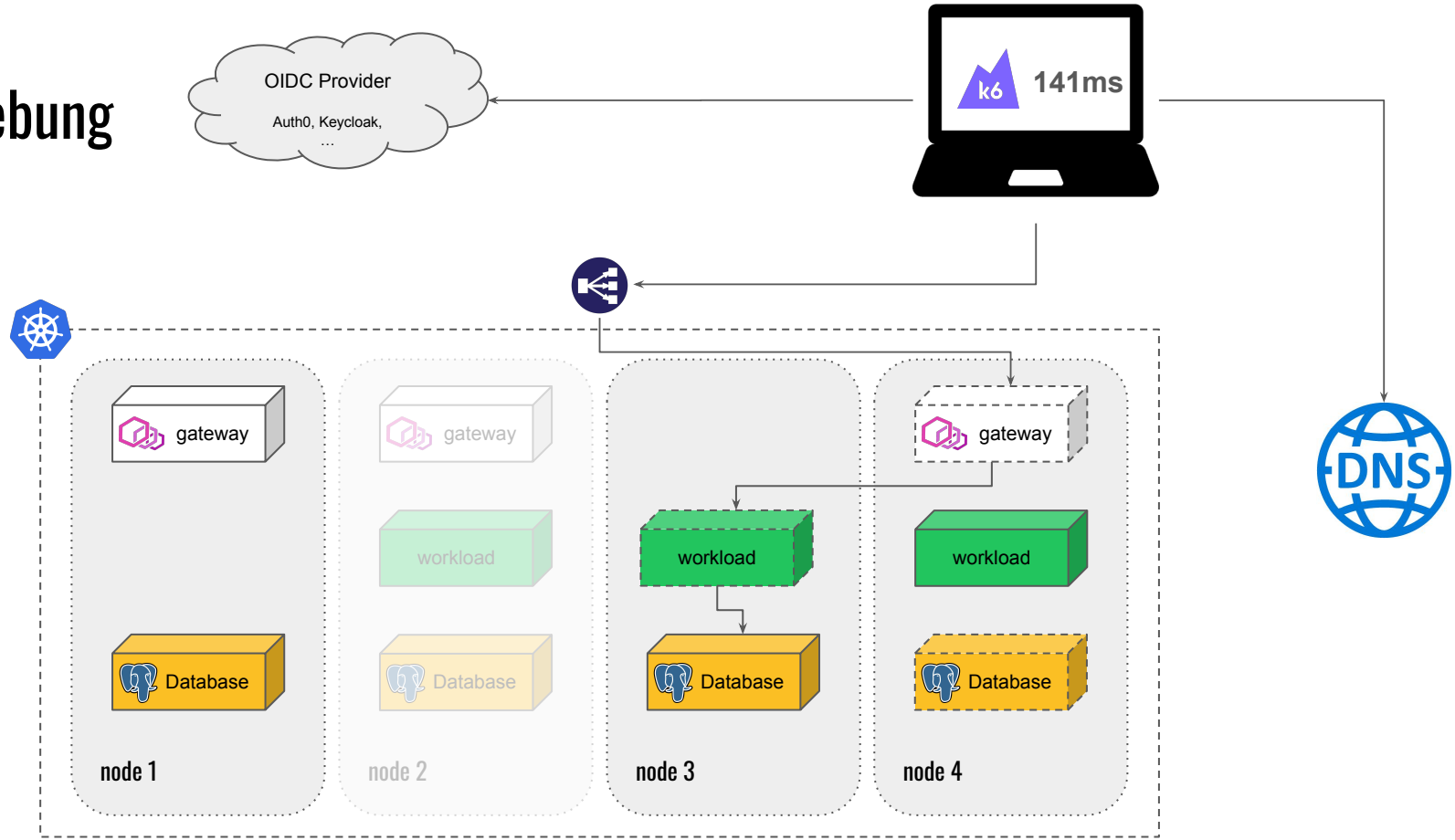




Testumgebung

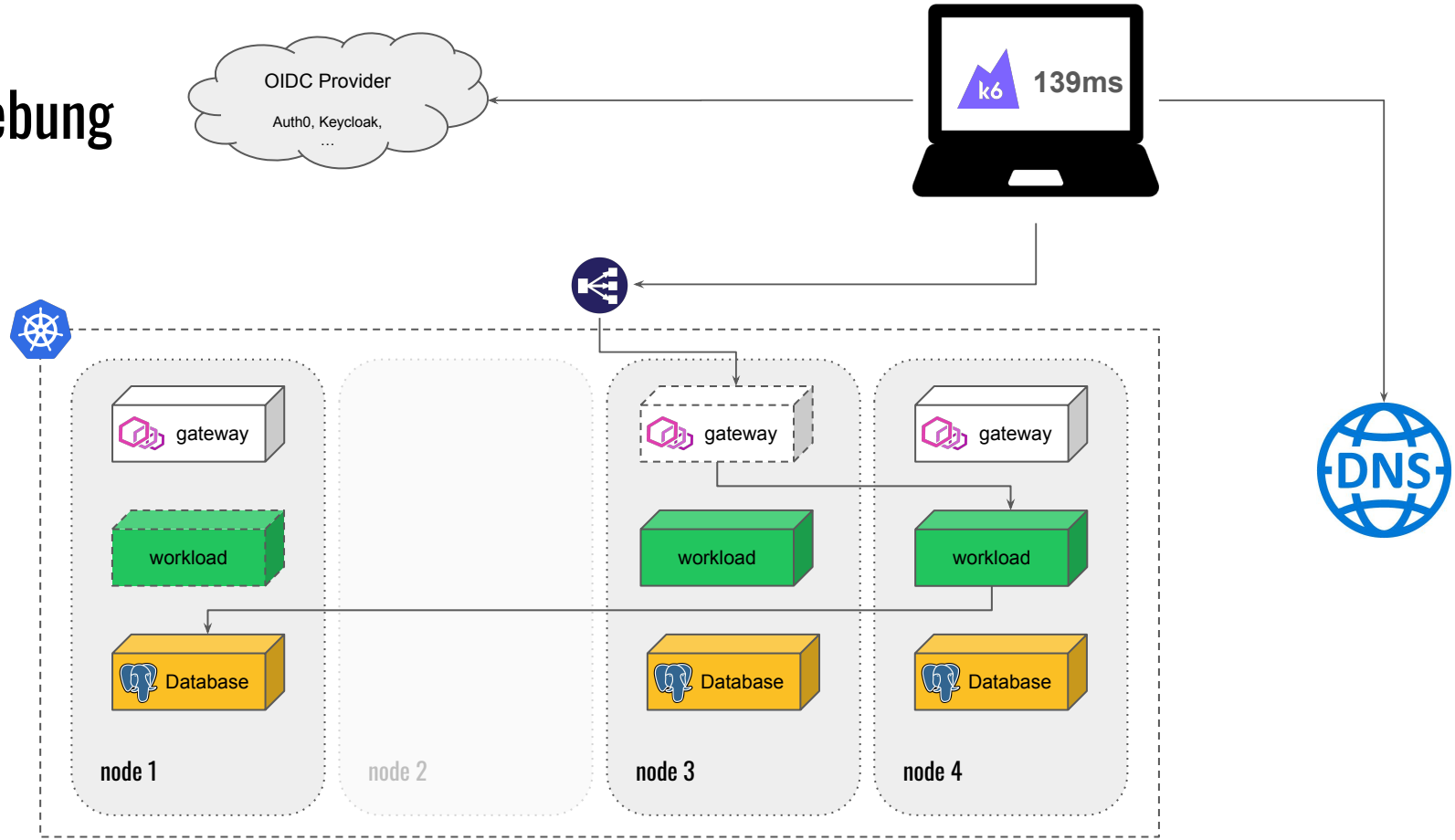


Testumgebung





Testumgebung

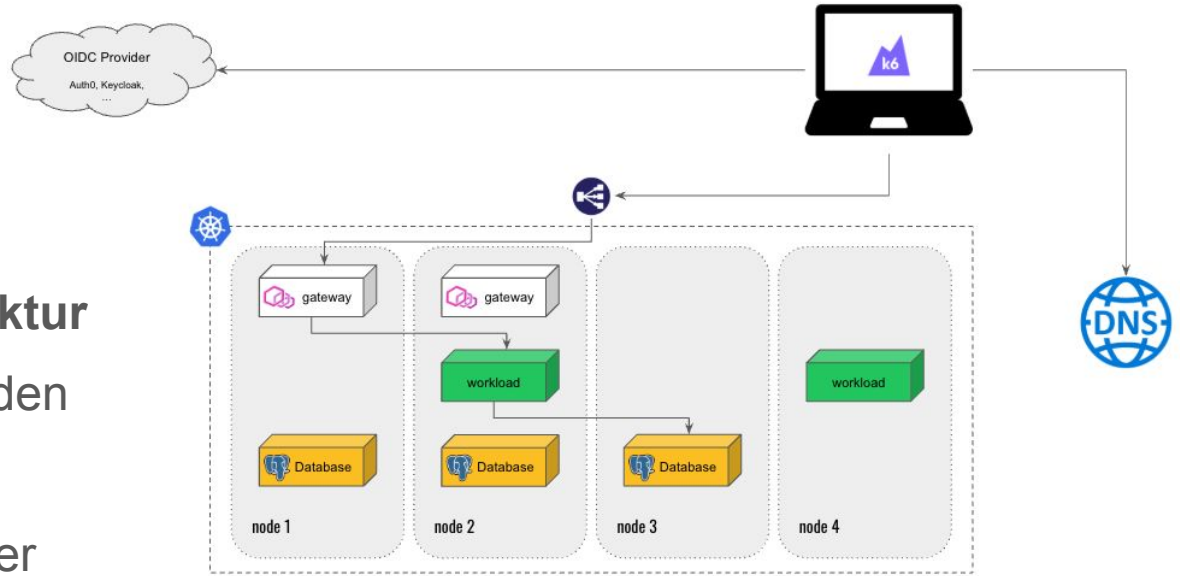




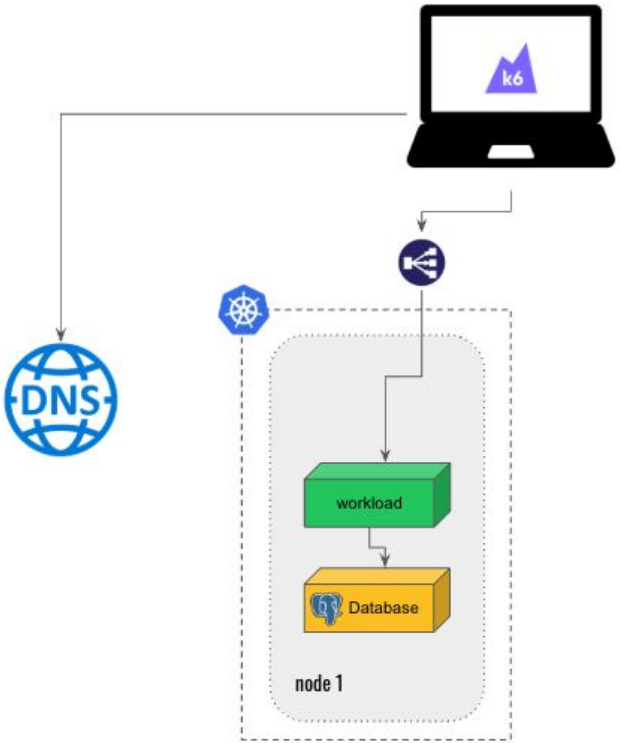
Testumgebung

End-to-end

- **Realitätsnah**
- **Kundenperspektive**
- Testet komplette **Infrastruktur**
- **Abhängige Services** werden mitgetestet
- **kein gezieltes Testen** einer einzelnen Komponente



Testumgebung





Laborumgebung

- z.B. **Single-Pod** Perspektive
- Ermittlung der **resource usage**
- Experimentieren mit Werten für **resource requests** und **limits**
- Nur **notwendige Abhängigkeiten** werden mit getestet
- z.B. **Single-Node** Cluster **ohne autoscaling**

Anforderungen an einen Load Generator

Protokoll-Vielfalt

Support für HTTP/S, WebSockets, gRPC, MQTT, TCP, …

Test-as-Code & CI/CD

Definition von Tests als Code (z.B. JavaScript, Java, Python, …) und einfache Einbindung in Pipelines.

Verteilte Last

Native Fähigkeit, Last über mehrere Worker-Nodes oder Cluster hinweg zu skalieren.

Flexibles Test-Modeling

Steuerung von Virtual Users (VU), Requests/s, Ramp-up/down & Szenarien (z.B. per HAR Files)

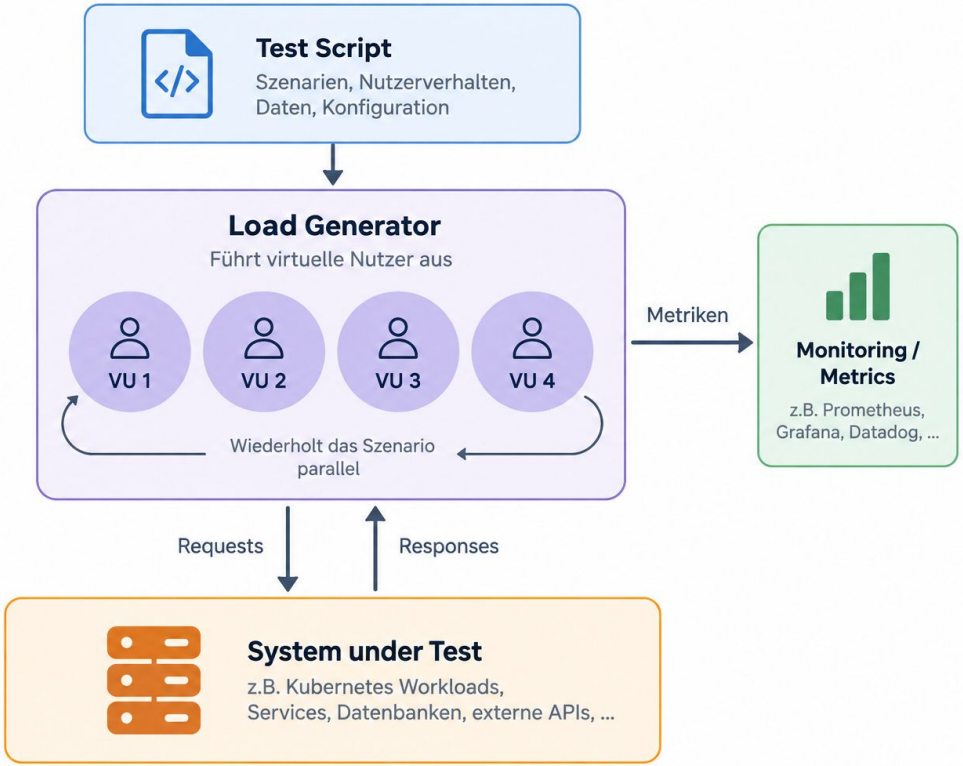
Metriken

Streaming von Live-Daten (Prometheus, InfluxDB) und detaillierte Berichte.

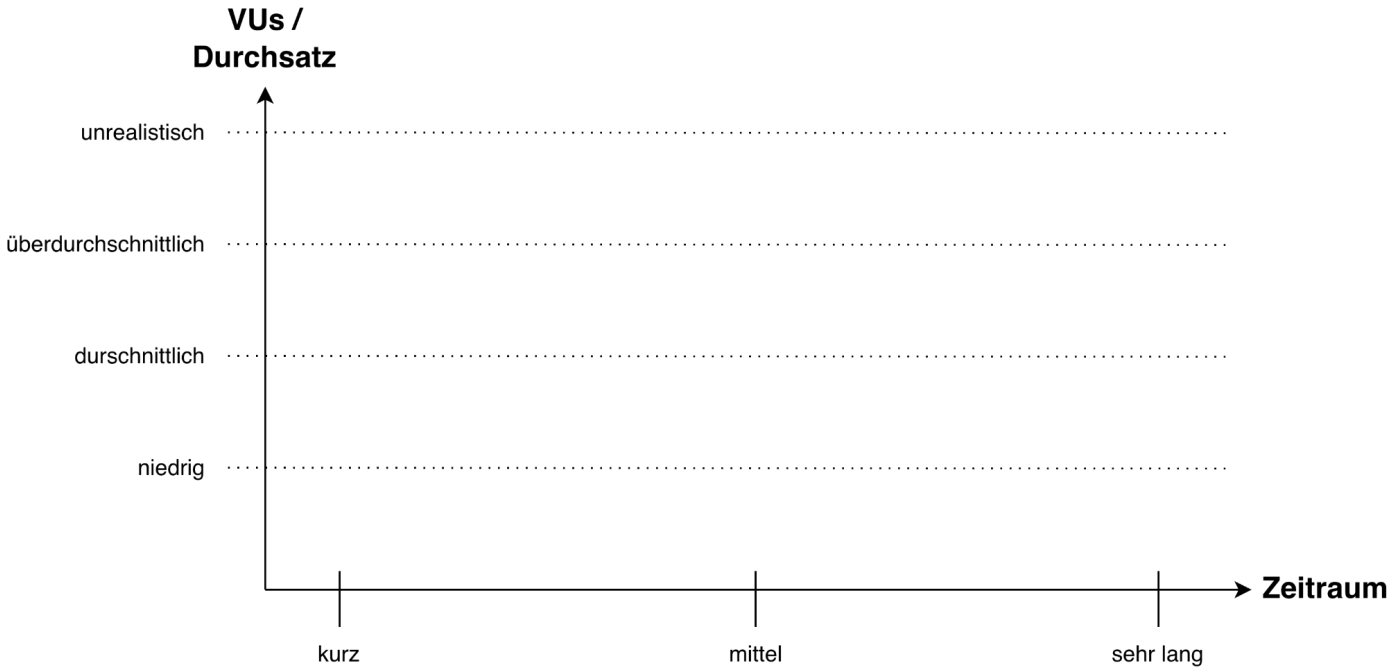
Erweiterbarkeit

Einfaches Nachladen von Drittanbieter-Bibliotheken oder Custom-Protokollen über Plugins.

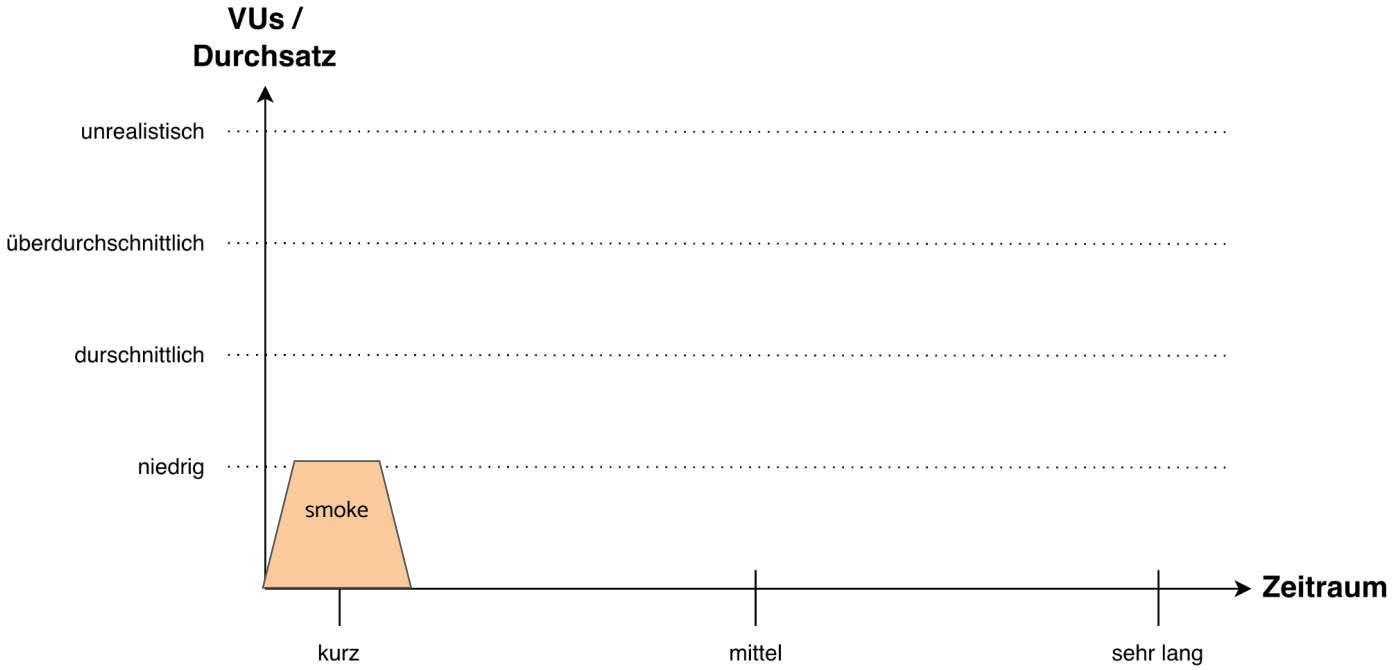
Wie funktioniert ein Load Generator?



Arten von Lasttests



Smoke Test

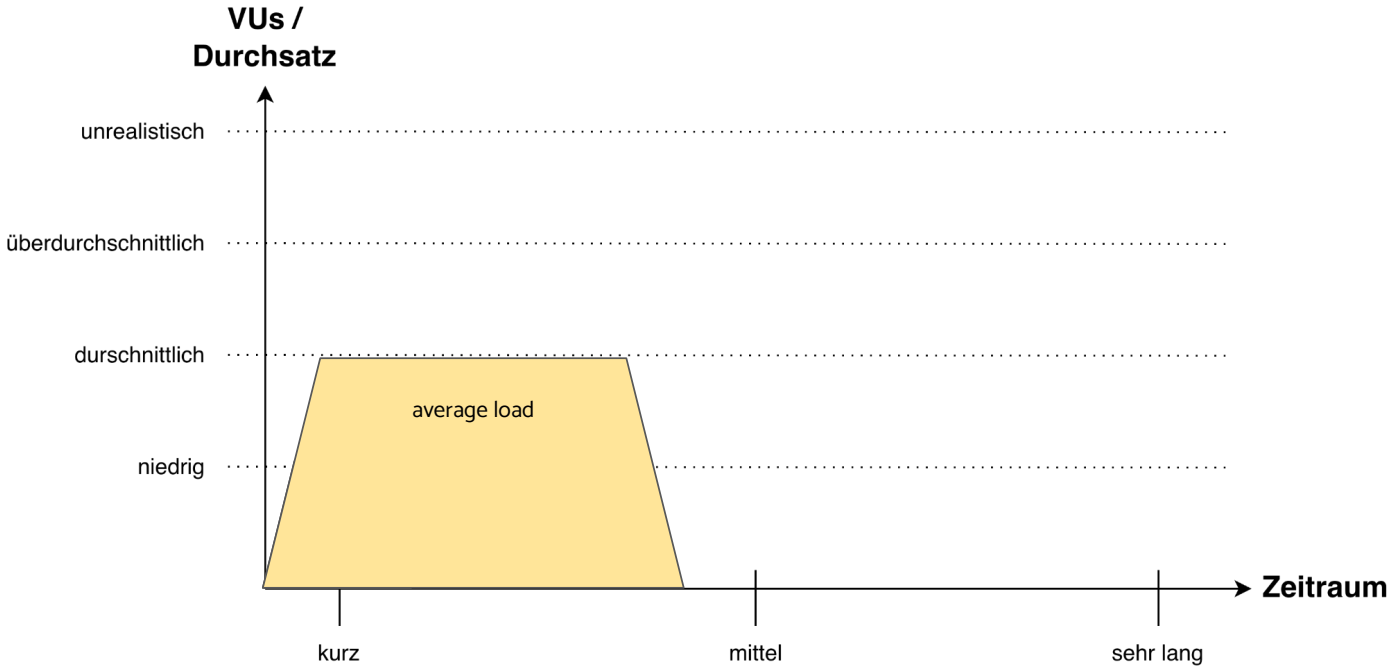


Minimaler Lasttest

Validieren eines Test-Szenarios

Einbinden in Pipeline zum Aufdecken von Regressionen

Average Load Test



Alltagslast

Ermitteln der Baseline
- CPU Verbrauch
- RAM Verbrauch
- Antwortzeit

Stabiler Betrieb (SLA)

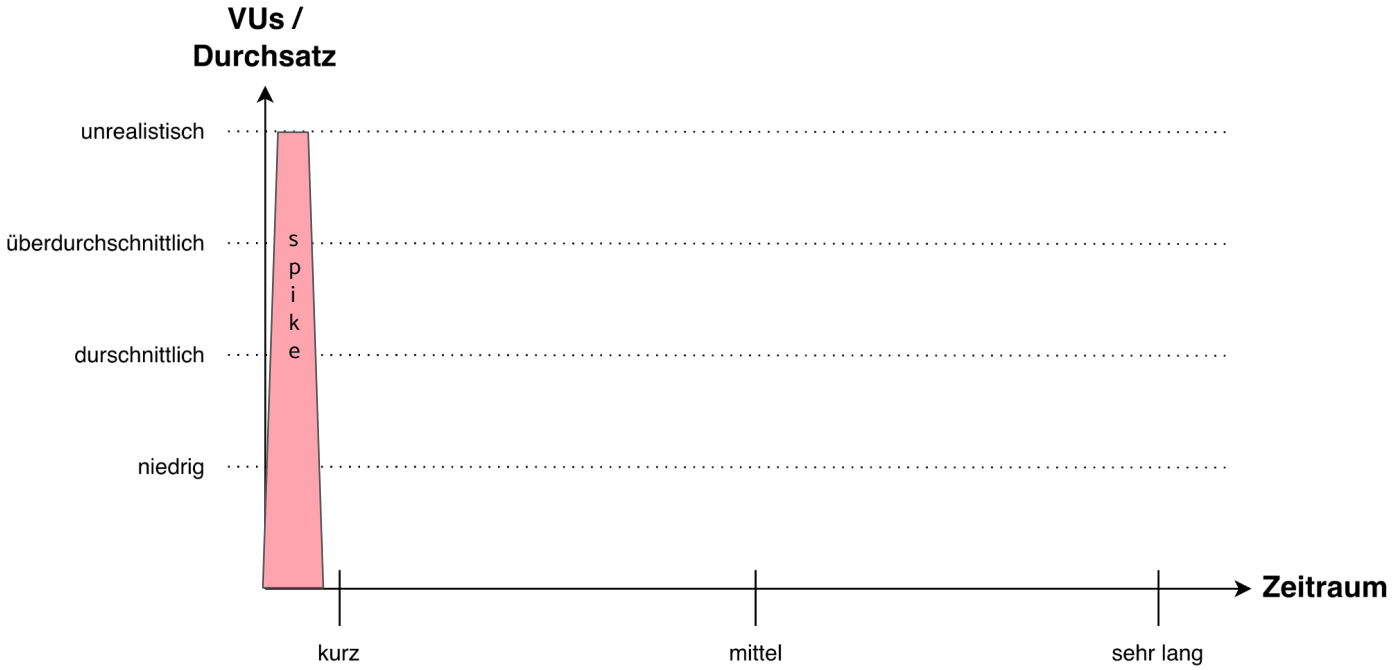
Stress Test



Black Friday Test (1/2)

Test des autoscalings
und der Resilienz

Spike Test

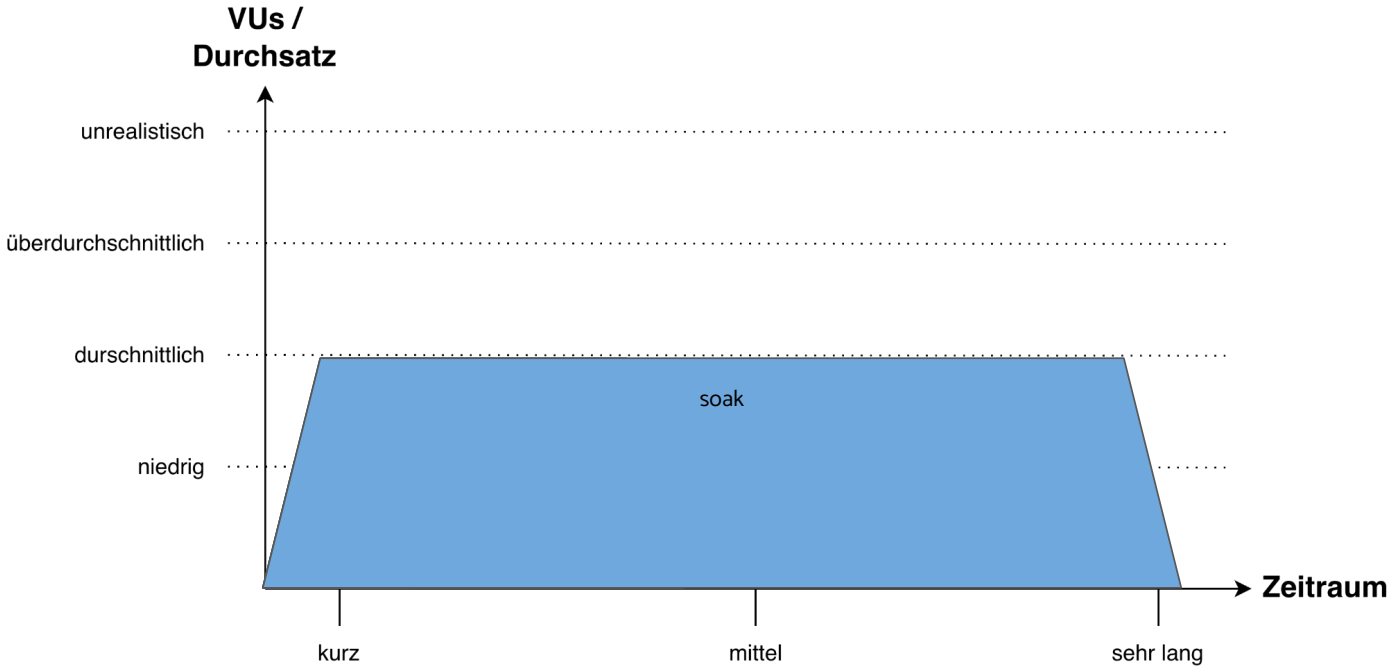


Black Friday Test (2/2)

Scale up/down
- HPA
- Cluster autoscaler

Resilienz
- Ausfall?
- Wie lange? (MTTR)
- Blast Radius

Soak Test



Langzeittest (z.B. 24h)

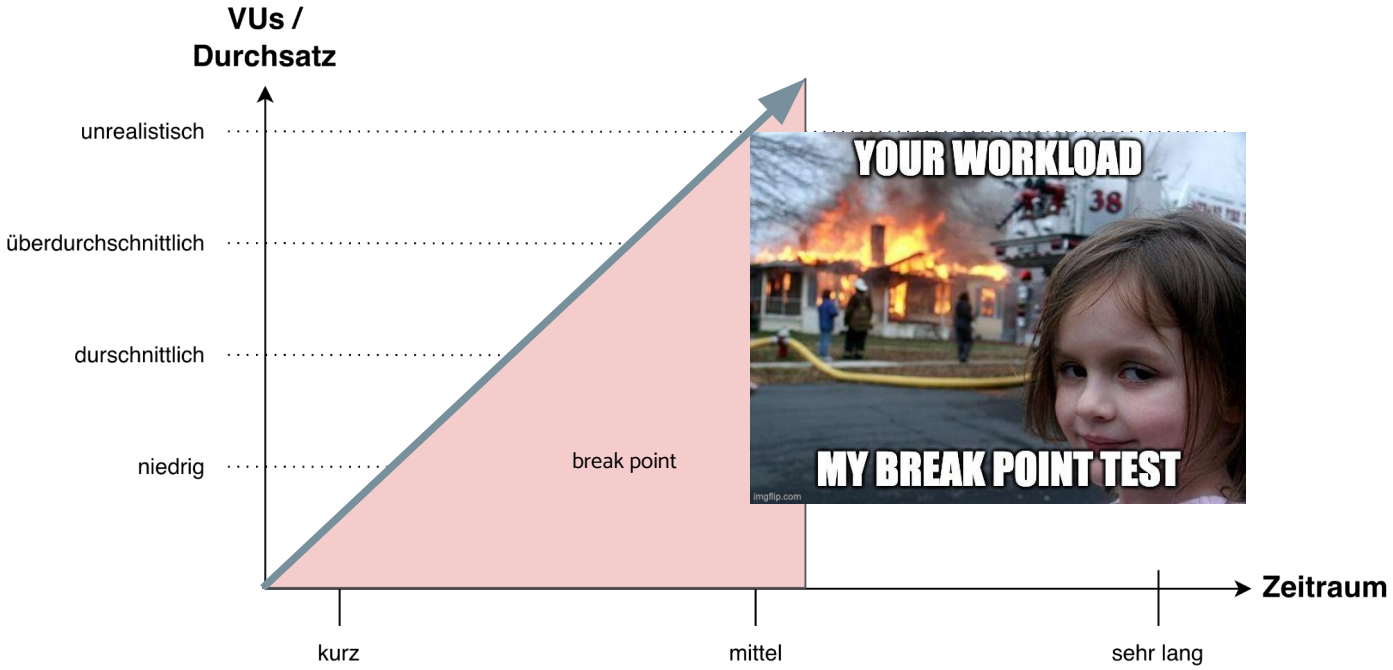
Ermitteln von Langzeit Effekten:

- Memory Leaks
- Connection Pool Exhaustion
- Performance Degradation
- Disks filling up (Logs / Database)

Verhalten während Maintenance Tasks:

- Upgrades
- Migrationen

Break Point Test



Erhöhung bis zum Kollaps

Ermitteln von:

- max. Kapazität
- Bottlenecks

Was ist der richtige Test / die richtige Testumgebung für mich?



It depends…

Welche individuellen Fragen sollen beantwortet werden?



Kombination

Selten kann ein einzelner Test alle relevanten Fragen direkt beantworten



Evolutionär

Im Laufe der Zeit werden sich Ihre Fragen ändern.

Testbeispiel

```
import { check, sleep } from "k6";
import http from "k6/http";

export let options = {
  stages: [
    { duration: "30s", target: 10 },    // ramp up
    { duration: "3m",  target: 10 },    // main
    { duration: "30s", target: 0 },     // ramp down
  ]
};

export default function() {
  let res = http.get("https://iits-consulting.de");
  check(res, {
    "is status 200": (r) => r.status === 200
  });
  sleep(1);
};
```

```
~/Projekte/performance-talk
☺  k6 cloud run ./basic-load.js
```




```
WARN[0000] No projectID specified, using default project of the https://stearz.grafana.net stack

     execution: cloud
        script: ./basic-load.js
        output: https://stearz.grafana.net/a/k6-app/runs/8523080

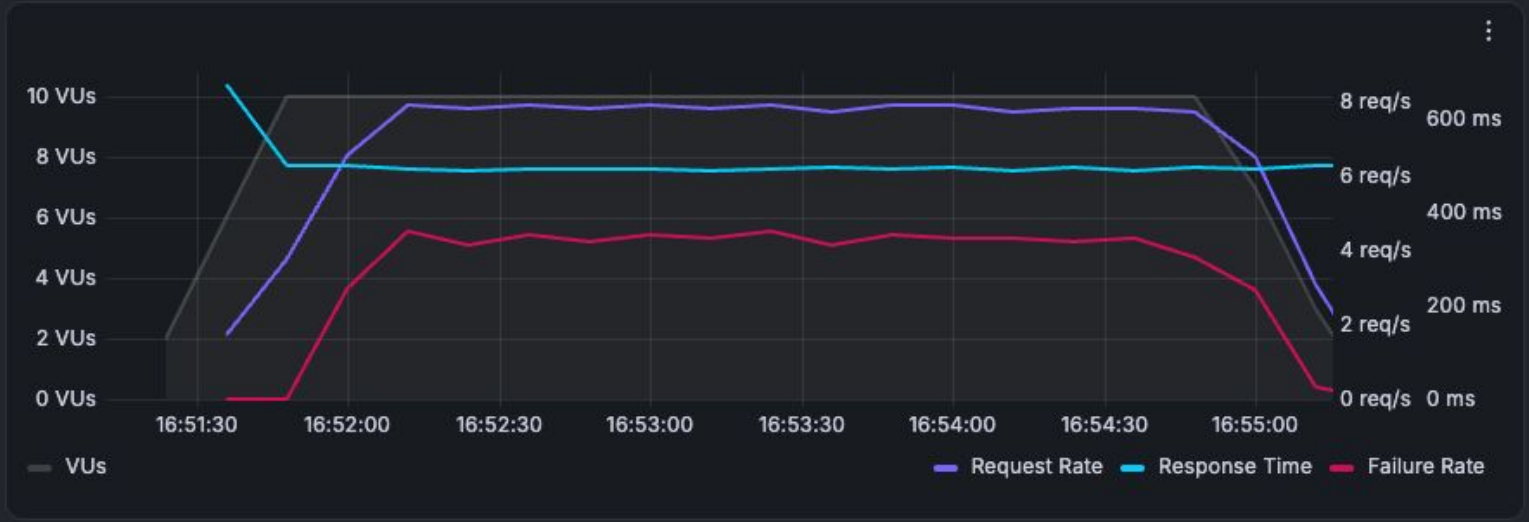
     scenarios: (100.00%) 1 scenario, 10 max VUs, 4m30s max duration (incl. graceful stop):
              * default: Up to 10 looping VUs for 4m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)


Run    [=======>------------------------------] 0m57.7s/4m30s
```

PERFORMANCE OVERVIEW

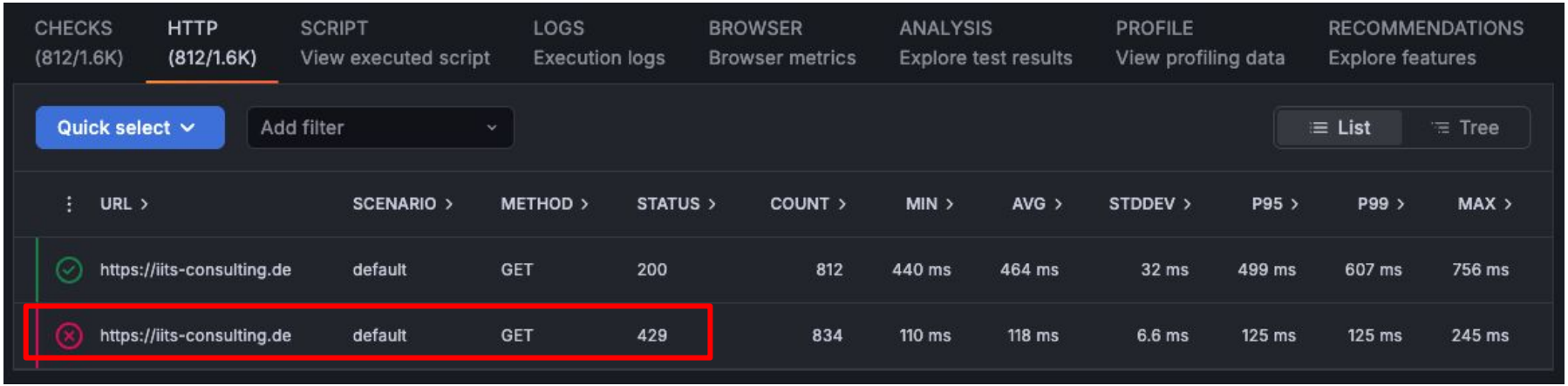
The 95th percentile response time of the system being tested was 495 ms, and 1 646 requests were made with 834 failures at an average rate of 6.8 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.6K reqs	834 reqs	7.92 reqs/s	495 ms



SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one



HTTP/1.1 429 Too Many Requests

Gelernte Lektionen (1/3)

Zugriff auf Monitoring Systeme notwendig:

- Alerts
- Metrics
- Logs
- Traces



Gelernte Lektionen (2/3)

Adjust Test Environment

Sometimes test results require adjusting the test environment while testing

Self-Managed Kubernetes

If you manage Kubernetes yourself: Set `kubeReserved` and `systemReserved` before any testing

I/O Caching

Leave some free RAM on the node for I/O Caching

It’s still hardware (somewhere)

Sometimes 2 Cores < 1 Core (Node types matter)

Gelernte Lektionen (3/3)

Kommunikation

Lasttest vorher mit relevanten Abteilungen und Dienstleistern abstimmen.

Fokus behalten

Nie mehr als eine Änderung gleichzeitig vornehmen.

Dokumentation

Jede Veränderung und das dazugehörige Ergebnis dokumentieren.

Reproduzierbarkeit

Tests wiederholen, um zufällige äußere Faktoren auszuschließen.

Root Cause Analysis

Wenn etwas bricht, die Ursache(n) finden, dokumentieren und dann reparieren

Realistische Erwartung

Dinge werden und müssen kaputt gehen.
Das ist der Weg.

Warum Lasttests unbeliebt sind...

Team-Expertise

Erfordert teamübergreifende Expertise und breites Fachwissen.

Geringes Momentum

Motivation eines Teams reicht selten aus, um andere Teams zu aktivieren.

Lokale Illusion

"Works on my machine"

Aufwand

Die Durchführung kostet Geld und Zeit.

Mehr als nur Tech

Lasttests sind kein ausschließlich technisches Projekt.

Nicht trivial

Komplexe Szenarien erfordern Planung und Analyse.

Warum Lasttests unbeliebt sind... - *...aber notwendig!*

**“Wer nicht selbst testet,
lässt seine Kunden testen.”**



Do or do not.
There is no try.





Vielen Dank

stephan.schwarz@iits-consulting.de